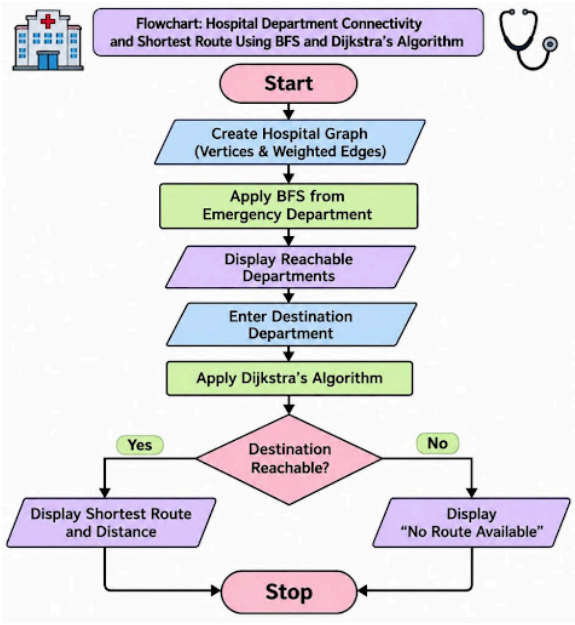
Title : Hospital Department Connectivity and Shortest Route Using BFS and Dijkstra's Algorithm in C++

Name: Vaishali Rathod | Roll No.: 26013 | Branch: ENTC | College: Amrutvahini College of Engineering, Sangamner

Problem Statement :

Develop a C++ program to represent hospital departments as vertices and corridors as weighted edges. Apply BFS from the emergency department to display reachable units and apply Dijkstra's algorithm to calculate the shortest route and distance to a selected department.

Flowchart :



Flowchart: Hospital Department Connectivity and Shortest Route Using BFS and Dijkstra's Algorithm

Code :

```
#include <iostream>
#include <vector>
#include <string>
#include <queue>
#include <limits>
#include <iomanip>
#include <algorithm>

using namespace std;

const int INF = numeric_limits<int>::max();

struct Edge {
    int to;
    int weight;
};

class HospitalGraph {
private:
    int numDepartments;
    vector<string> deptNames;
    vector<vector<Edge>> adjList;
```

```
public:
    HospitalGraph(int n) : numDepartments(n), deptNames(n), adjList(n) {}

    void setDepartmentName(int id, const string& name) {
        if (id >= 0 && id < numDepartments) {
            deptNames[id] = name;
        }
    }

    void addCorridor(int u, int v, int distance) {
        if (u >= 0 && u < numDepartments && v >= 0 && v < numDepartments && distance > 0) {
            adjList[u].push_back({v, distance});
            adjList[v].push_back({u, distance});
        }
    }

    void displayGraph() const {
        cout << "\n--- Hospital Department Corridor Network ---\n";
        for (int i = 0; i < numDepartments; ++i) {
            cout << "[" << i << "] " << left << setw(22) << deptNames[i] << " -> ";
            if (adjList[i].empty()) {
                cout << "No connected corridors";
            } else {
                for (size_t j = 0; j < adjList[i].size(); ++j) {
                    cout << deptNames[adjList[i][j].to] << " (" << adjList[i][j].weight << "m)";
                    if (j + 1 < adjList[i].size()) cout << ", ";
                }
            }
            cout << "\n";
        }
    }

    // Apply BFS from Emergency Department (vertex 0) to display reachable units
    void bfsFromEmergency(int startDept = 0) const {
        if (startDept < 0 || startDept >= numDepartments) {
            cout << "Invalid starting department ID.\n";
            return;
        }

        vector<bool> visited(numDepartments, false);
        queue<int> q;

        visited[startDept] = true;
        q.push(startDept);

        cout << "\n--- BFS Traversal from " << deptNames[startDept] << " ---\n";
        cout << "Reachable Hospital Units in Level Order:\n";

        int step = 1;
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            cout << "  " << step++ << ". [" << curr << "] " << deptNames[curr] << "\n";

            for (const auto& edge : adjList[curr]) {
                if (!visited[edge.to]) {
                    visited[edge.to] = true;
                    q.push(edge.to);
                }
            }
        }
    }

    // Apply Dijkstra's algorithm to calculate shortest route and distance
    void shortestRouteDijkstra(int startDept, int destDept) const {
```

```
        if (startDept < 0 || startDept >= numDepartments || destDept < 0 || destDept >=
numDepartments) {
            cout << "Invalid department ID selected.\n";
            return;
        }

        vector<int> dist(numDepartments, INF);
        vector<int> parent(numDepartments, -1);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

        dist[startDept] = 0;
        pq.push({0, startDept});

        while (!pq.empty()) {
            auto [d, u] = pq.top();
            pq.pop();

            if (d > dist[u]) continue;

            for (const auto& edge : adjList[u]) {
                if (dist[u] != INF && dist[u] + edge.weight < dist[edge.to]) {
                    dist[edge.to] = dist[u] + edge.weight;
                    parent[edge.to] = u;
                    pq.push({dist[edge.to], edge.to});
                }
            }
        }

        cout << "\n--- Dijkstra's Shortest Path Result ---\n";
        cout << "Source      : [" << startDept << "] " << deptNames[startDept] << "\n";
        cout << "Destination : [" << destDept << "] " << deptNames[destDept] << "\n";

        if (dist[destDept] == INF) {
            cout << "Status      : No Route Available (Destination is unreachable).\n";
            return;
        }

        vector<int> path;
        for (int v = destDept; v != -1; v = parent[v]) {
            path.push_back(v);
        }
        reverse(path.begin(), path.end());

        cout << "Shortest Route : ";
        for (size_t i = 0; i < path.size(); ++i) {
            cout << deptNames[path[i]];
            if (i + 1 < path.size()) cout << " -> ";
        }
        cout << "\nTotal Distance : " << dist[destDept] << " meters\n";
    }

    void listDepartments() const {
        cout << "\nAvailable Hospital Departments:\n";
        for (int i = 0; i < numDepartments; ++i) {
            cout << "  " << i << " : " << deptNames[i] << "\n";
        }
    }
};

int main() {
    HospitalGraph hospital(6);

    hospital.setDepartmentName(0, "Emergency Department");
    hospital.setDepartmentName(1, "Radiology (X-Ray/CT)");
    hospital.setDepartmentName(2, "ICU");
    hospital.setDepartmentName(3, "Operation Theatre");
```

```
    hospital.setDepartmentName(4, "Pathology Lab");
    hospital.setDepartmentName(5, "Pharmacy");

    // Preloaded hospital corridors with distances in meters
    hospital.addCorridor(0, 1, 30);
    hospital.addCorridor(0, 2, 50);
    hospital.addCorridor(1, 2, 15);
    hospital.addCorridor(1, 4, 25);
    hospital.addCorridor(2, 3, 20);
    hospital.addCorridor(4, 3, 35);
    hospital.addCorridor(4, 5, 20);
    hospital.addCorridor(3, 5, 15);

    int choice, dest;
    do {
        cout << "\n============================================\n";
        cout << "   HOSPITAL DEPARTMENT GRAPH NAVIGATION\n";
        cout << "============================================\n";
        cout << "1. Display Hospital Graph (Departments & Corridors)\n";
        cout << "2. Apply BFS from Emergency Department\n";
        cout << "3. Find Shortest Route using Dijkstra's Algorithm\n";
        cout << "4. Exit\n";
        cout << "Choice: ";
        if (!(cin >> choice)) {
            cout << "Invalid choice.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        switch (choice) {
            case 1:
                hospital.displayGraph();
                break;
            case 2:
                hospital.bfsFromEmergency(0);
                break;
            case 3:
                hospital.listDepartments();
                cout << "Enter Destination Department ID (0 to 5): ";
                cin >> dest;
                hospital.shortestRouteDijkstra(0, dest);
                break;
            case 4:
                cout << "Exiting...\n";
                break;
            default:
                cout << "Invalid choice.\n";
        }
    } while (choice != 4);

    return 0;
}
```

Output :

The following console execution screenshots show the complete output of the program for displaying the hospital department corridor graph, running BFS traversal from the Emergency Department, and calculating the shortest route and distance using Dijkstra's algorithm:

```
==========================================
   HOSPITAL DEPARTMENT GRAPH NAVIGATION
==========================================
1. Display Hospital Graph (Departments & Corridors)
2. Apply BFS from Emergency Department
3. Find Shortest Route using Dijkstra's Algorithm
4. Exit
Choice: 1

--- Hospital Department Corridor Network ---
[0] Emergency Department   -> Radiology (X-Ray/CT) (30m), ICU (50m)
[1] Radiology (X-Ray/CT)   -> Emergency Department (30m), ICU (15m), Pathology Lab (25m)
[2] ICU                    -> Emergency Department (50m), Radiology (X-Ray/CT) (15m), Operation Theatre (20m)
[3] Operation Theatre      -> ICU (20m), Pathology Lab (35m), Pharmacy (15m)
[4] Pathology Lab          -> Radiology (X-Ray/CT) (25m), Operation Theatre (35m), Pharmacy (20m)
[5] Pharmacy               -> Pathology Lab (20m), Operation Theatre (15m)

Choice: 2
--- BFS Traversal from Emergency Department ---
Reachable Hospital Units in Level Order:
  1. [0] Emergency Department
  2. [1] Radiology (X-Ray/CT)
  3. [2] ICU
  4. [4] Pathology Lab
  5. [3] Operation Theatre
  6. [5] Pharmacy
```

Console Output 1: Hospital Corridor Network Display and BFS Reachable Units from Emergency Department

```
Choice: 3
Available Hospital Departments:
  0 : Emergency Department
  1 : Radiology (X-Ray/CT)
  2 : ICU
  3 : Operation Theatre
  4 : Pathology Lab
  5 : Pharmacy
Enter Destination Department ID (0 to 5): 3

--- Dijkstra's Shortest Path Result ---
Source         : [0] Emergency Department
Destination    : [3] Operation Theatre
Shortest Route : Emergency Department -> Radiology (X-Ray/CT) -> ICU -> Operation Theatre
Total Distance : 65 meters

Choice: 3
Enter Destination Department ID (0 to 5): 5

--- Dijkstra's Shortest Path Result ---
Source         : [0] Emergency Department
Destination    : [5] Pharmacy
Shortest Route : Emergency Department -> Radiology (X-Ray/CT) -> Pathology Lab -> Pharmacy
Total Distance : 75 meters

Choice: 4
Exiting...
```

Console Output 2: Dijkstra's Shortest Route and Distance to Operation Theatre (65m) and Pharmacy (75m)

Conclusion :

1. Thus, we have implemented a menu-driven C++ program that models hospital departments as graph vertices and connecting corridors as weighted edges using an adjacency list representation.

2. Breadth-First Search (BFS) starting from the Emergency Department (vertex 0) traverses and displays all reachable hospital departments in level-by-level order in **O(V + E)** time complexity.

3. Dijkstra's shortest-path algorithm using a min-priority queue computes the minimum corridor distance and reconstructs the exact transfer path from the Emergency Department to any selected destination unit (such as 65 meters to the Operation Theatre and 75 meters to the Pharmacy) in **O((V + E) log V)** time complexity.